

MAINCRAFTS TECHNOLOGY

# Exploitation & Web Application Security Testing Report

Task-4: Exploitation & Web Application Security Testing

Target Application: Damn Vulnerable Web Application (DVWA)

Deployment: Dockerized container on Kali Linux host (127.0.0.1:8888)

Classification: Internal / Lab Use Only

Prepared by: SOUVIK DAS

Role: Cybersecurity Intern

# Table of Contents

---

- 1. Executive Summary .....3
- 2. Objective & Scope.....3
- 3. Methodology & Test Environment .....4
- 4. Concept Overview — OWASP Top Vulnerabilities.....4
- 5. Exploitation Findings.....5
  - 5.1 SQL Injection (SQLi).....5
  - 5.2 Cross-Site Scripting (XSS) .....7
  - 5.3 Directory & File Enumeration .....9
  - 5.4 Burp Suite — Interception & Parameter Tampering .....11
- 6. Impact Analysis .....13
- 7. Risk Rating Summary .....14
- 8. Remediation Recommendations .....14
- 9. Conclusion & Learning Outcomes.....15
- Appendix A: Evidence Index .....16

## 1. Executive Summary

---

This report documents the controlled exploitation and web application security testing carried out against Damn Vulnerable Web Application (DVWA), as Task-4 of the Maincrafts Technology Cybersecurity & Ethical Hacking internship. Building directly on the Task-3 Vulnerability Assessment Report — where vulnerabilities were identified but deliberately not exploited — this task validates those findings by demonstrating real, working exploitation techniques in a fully isolated, self-hosted lab environment.

Testing covered four core areas: SQL Injection, Cross-Site Scripting (both Reflected and Stored), hidden directory/file enumeration, and manual request interception/tampering using Burp Suite. Each exploitation attempt was successful, confirming that the vulnerabilities identified in Task-3 (and additional ones surfaced here) are not just theoretical, but practically exploitable under the application's default (Low) security configuration.

All testing was performed exclusively against the local DVWA Docker container (127.0.0.1:8888) on a Kali Linux host. No real-world systems, third-party services, or production applications were targeted at any point, in line with the strict ethical scope defined for this task.

| Risk Rating | Number of Findings | Examples                                               |
|-------------|--------------------|--------------------------------------------------------|
| Critical    | 0                  | —                                                      |
| High        | 2                  | SQL Injection authentication/data exposure, Stored XSS |
| Medium      | 2                  | Reflected XSS, exposed setup/config files              |
| Low         | 1                  | Request/parameter tampering via Burp Suite             |

## 2. Objective & Scope

---

Task-3 focused purely on reconnaissance: performing scans, identifying vulnerabilities, and documenting them without exploitation. Task-4 builds directly on this by requiring controlled, hands-on exploitation to validate which of those findings are real and how severe they are — a task core to the roles of Penetration Tester, Red Team Engineer, and Security Analyst.

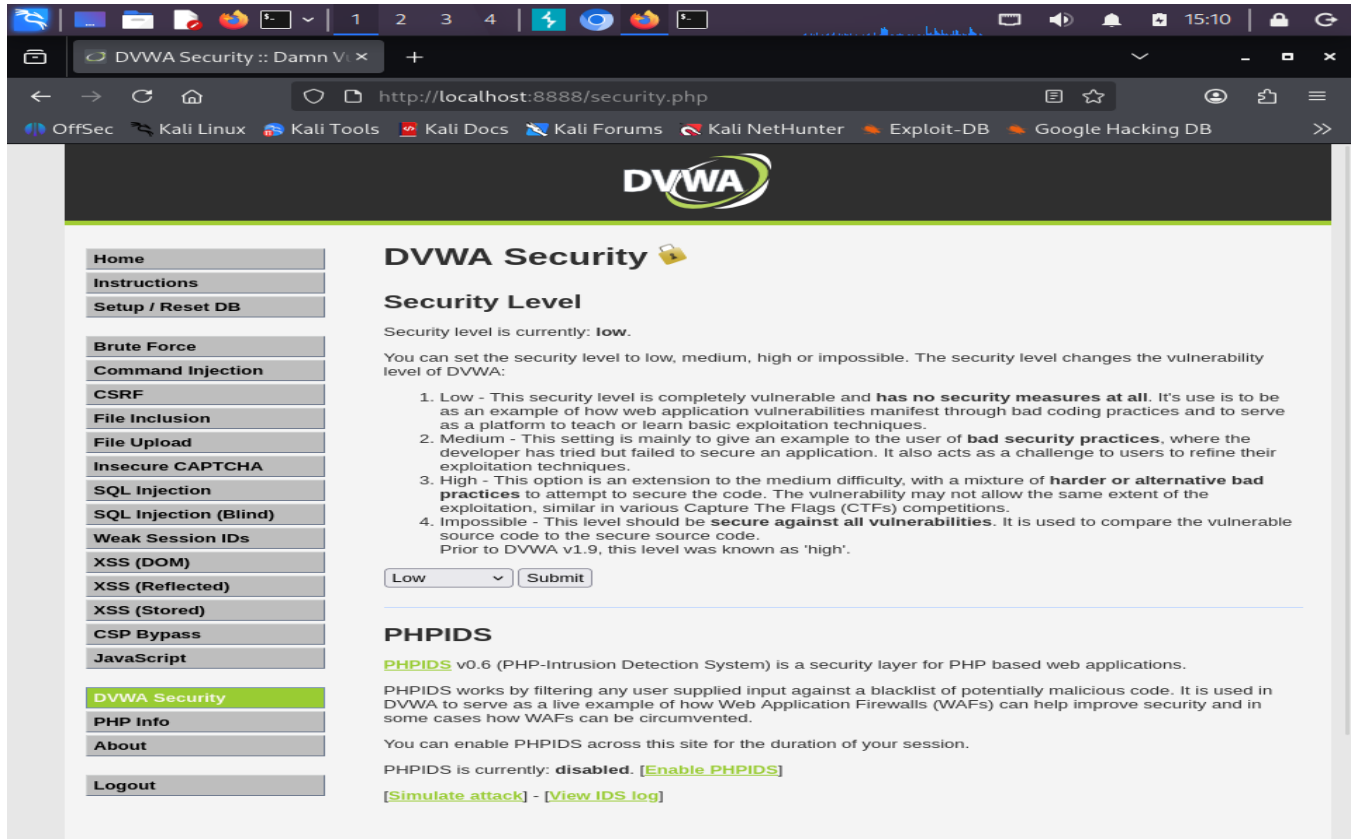
### 2.1 In-Scope

- DVWA (Damn Vulnerable Web Application) — local Docker container only
- SQL Injection testing
- Cross-Site Scripting (Reflected and Stored) testing
- Directory and hidden file enumeration
- Manual request interception and parameter tampering via Burp Suite

### 2.2 Out-of-Scope

- Any real-world or third-party website

- Denial-of-service or destructive testing
- Automated exploitation frameworks (e.g. Metasploit modules) — manual, guided exploitation only for this task



### 3. Methodology & Test Environment

For this experiment, DVWA was used as the target application, deployed via Docker on a Kali Linux host, consistent with the environment used in Task-3.

- Host operating system: Kali Linux (attacker/analyst workstation)
- Target application: DVWA (Damn Vulnerable Web Application), Docker image vulnerables/web-dvwa
- Target address: 127.0.0.1:8888 (container port 80 remapped to host port 8888, to free port 8080 for Burp Suite's default proxy listener)
- DVWA Security Level: Low (set via DVWA Security page) — required to reliably reproduce these vulnerabilities without additional input-validation obstacles
- Tools used: DVWA built-in vulnerability modules (browser-based), Dirsearch v0.4.3, Burp Suite Community Edition

### 4. Concept Overview — OWASP Top Vulnerabilities

Before performing hands-on exploitation, the following core web application vulnerability classes were reviewed, per the Part A requirement of this task. Each is described in terms of what it is, how it works, and its real-world impact.

## 4.1 SQL Injection (SQLi)

What it is: SQL Injection occurs when untrusted user input is inserted directly into a SQL query without proper sanitization or parameterization, allowing an attacker to alter the query's logic.

How it works: An attacker submits crafted input (e.g. ' OR '1'='1) into a form field. If the application concatenates this input directly into a SQL statement, the injected logic changes what the database returns — for example, turning a single-row lookup into a query that always evaluates true, returning all rows.

Real-world impact: SQLi can lead to authentication bypass, unauthorized data disclosure (including credentials), data modification or deletion, and in severe cases, full database or server compromise. It remains one of the most damaging and common vulnerability classes in production web applications.

## 4.2 Cross-Site Scripting (XSS)

What it is: XSS occurs when an application includes untrusted input in its output (HTML/JavaScript context) without proper encoding, allowing an attacker to run arbitrary script in a victim's browser.

How it works (Reflected): The malicious script is included in a request (e.g. a URL parameter) and immediately reflected back in the response, executing in the browser of whoever clicks the crafted link. How it works (Stored): The malicious script is saved server-side (e.g. in a comment or guestbook entry) and executes for every user who later views that stored content, without needing a crafted link.

Real-world impact: Session hijacking via cookie theft, credential phishing through fake login prompts, defacement, and — in the case of Stored XSS — mass compromise of any user who simply views the affected page, making it significantly more dangerous than Reflected XSS.

## 4.3 Command Injection

What it is: Command Injection occurs when an application passes unsanitized user input into a system shell command.

How it works: If user input reaches a function like `exec()` or `system()` without sanitization, an attacker can append shell metacharacters (e.g. `;` `ls`, `&&` `whoami`) to execute arbitrary operating system commands alongside the intended one.

Real-world impact: Full compromise of the underlying host — arbitrary file read/write, credential theft, lateral movement, and complete server takeover in the worst case.

## 4.4 Broken Authentication / Authentication Flaws

What it is: Weaknesses in how an application manages login, session tokens, or credential storage, allowing attackers to bypass authentication or hijack sessions.

How it works: Common causes include predictable session IDs, missing session expiration, weak password policies, and logic flaws that let SQLi or parameter tampering bypass the login check entirely (as demonstrated in Section 5.1 and 5.4 of this report).

Real-world impact: Unauthorized account access, privilege escalation, and impersonation of legitimate users — often the first step in a larger breach.

## 4.5 File Inclusion

**What it is:** A vulnerability where an application allows user input to control which file is loaded or executed by the server, without proper validation.

**How it works:** Local File Inclusion (LFI) lets an attacker reference unintended local files (e.g. path traversal to read `/etc/passwd`); Remote File Inclusion (RFI) lets an attacker force the server to load and execute a file from an attacker-controlled remote location.

**Real-world impact:** Disclosure of sensitive server files, source code exposure, and — in RFI cases — full remote code execution on the server.

## 5. Exploitation Findings

---

The following sections document each exploitation activity performed against the DVWA lab target, following the structure required by this task: vulnerability description, exploitation steps taken, supporting evidence (screenshot placeholders marked for insertion), risk rating, and remediation guidance.

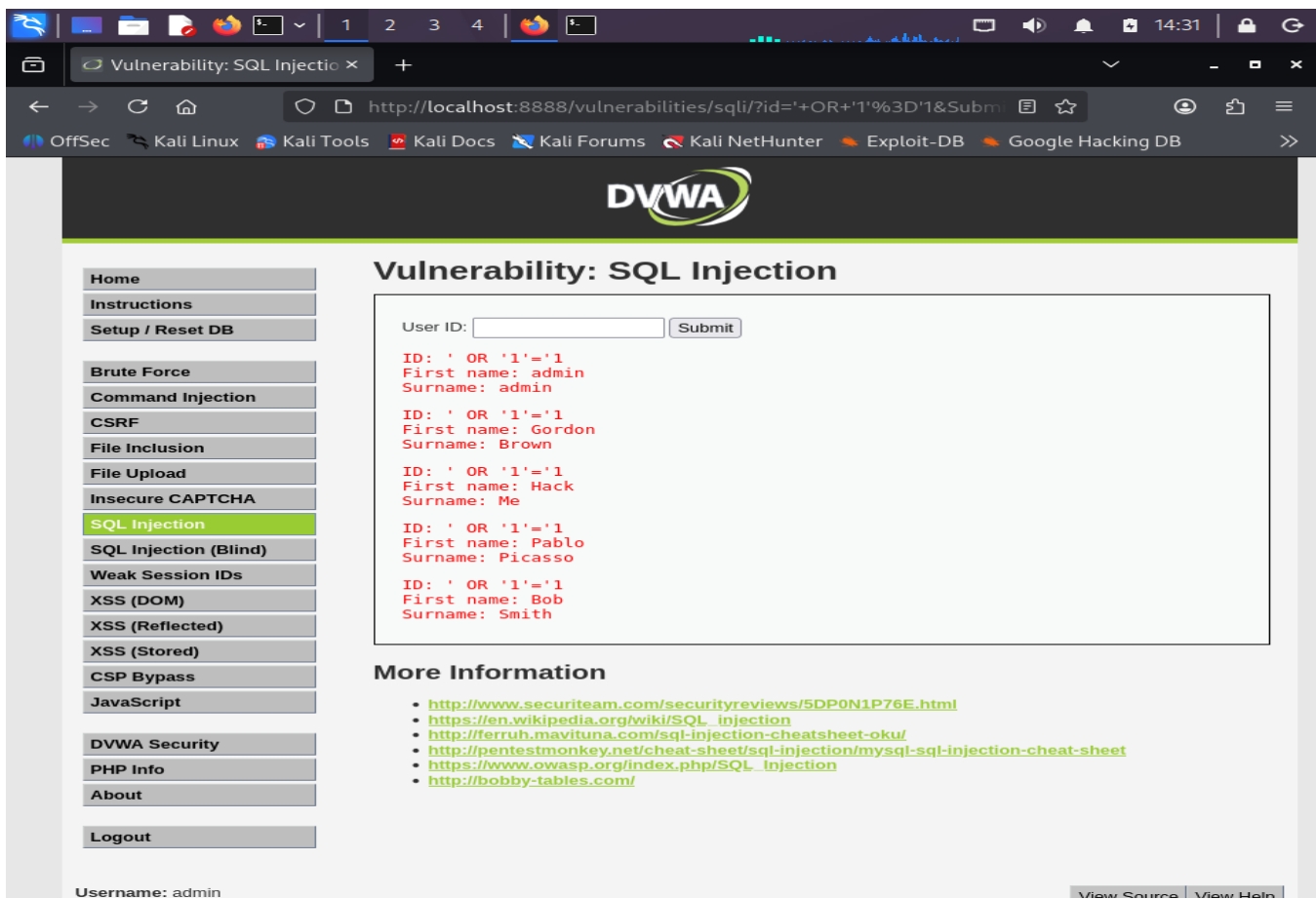
### 5.1 SQL Injection (SQLi)

#### **Finding 1: Authentication Logic Bypass & Data Enumeration via Boolean-Based SQL Injection** [High]

**Vulnerability:** The DVWA SQL Injection module (`/vulnerabilities/sqli/`) accepts a User ID value and inserts it directly into a backend SQL query without sanitization or parameterized queries.

**Exploitation Steps:** 1) Navigated to the SQL Injection module. 2) In the "User ID" field, submitted the payload: `' OR '1'='1`  
3) The application returned every record in the users table (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith) instead of a single matching user, confirming the injected logic (`' OR '1'='1`) short-circuited the intended WHERE clause to always evaluate true.

**Observed Evidence:** Browser URL reflected the raw payload in the id parameter (`?id='+OR+'1'%3D'1&Submit=Submit#`), and the response body listed all five user records in the database.



**Risk Level:** High — this confirms the application is vulnerable to classic boolean-based SQL Injection. In a real deployment this could be extended (e.g. via UNION-based queries) to extract credentials, other tables, or entire database contents.

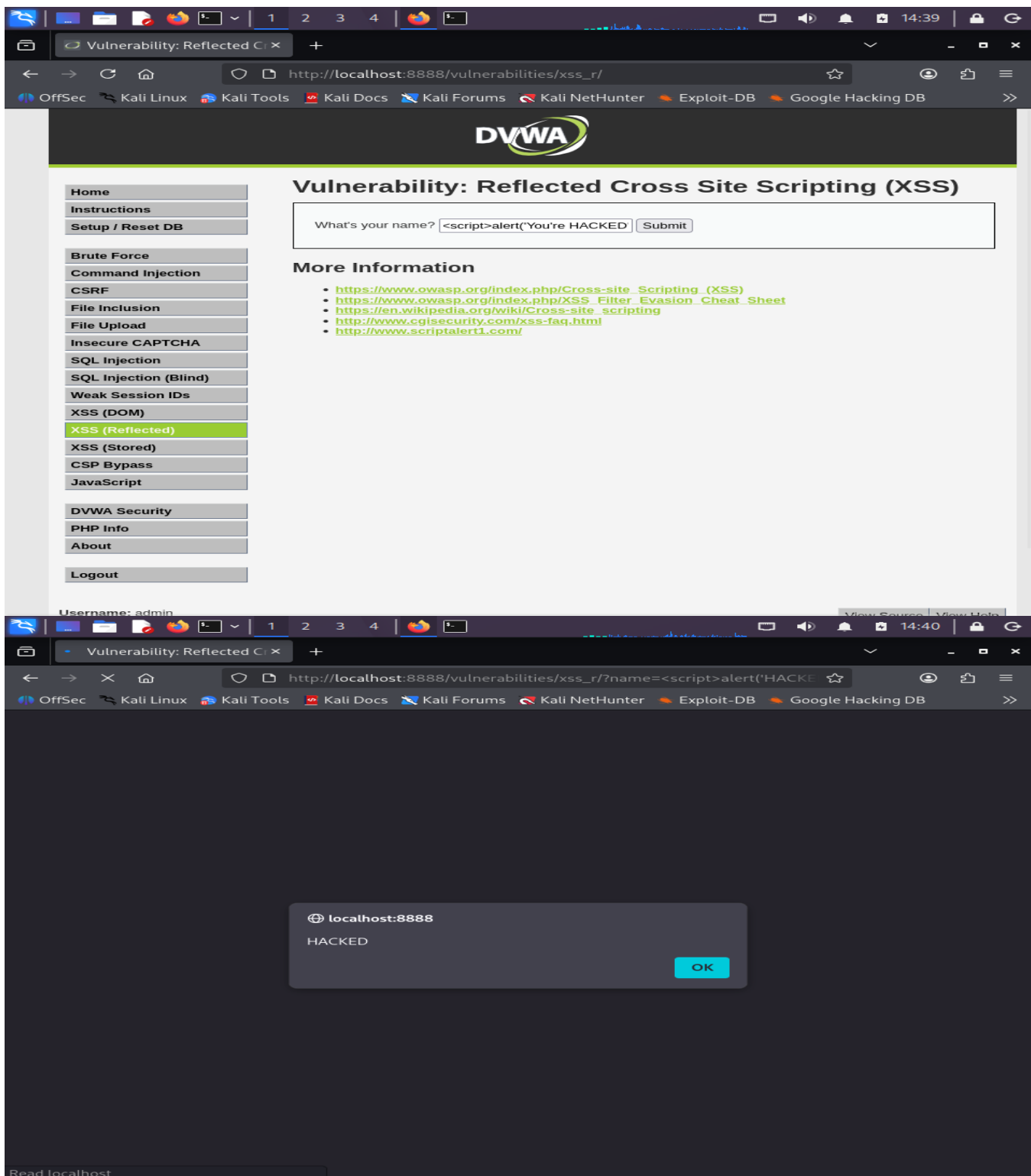
**Fix Recommendation:** Use parameterized queries / prepared statements (or an ORM with built-in escaping) for all database access — never concatenate raw user input into SQL strings. Additionally apply strict input validation and the principle of least privilege for the database account used by the application.

## 5.2 Cross-Site Scripting (XSS)

### Finding 2: Reflected Cross-Site Scripting [Medium]

**Vulnerability:** The DVWA Reflected XSS module (`/vulnerabilities/xss_r/`) echoes the "name" input parameter directly back into the page's HTML response without output encoding.

**Exploitation Steps:** 1) Navigated to the XSS (Reflected) module. 2) In the "What's your name?" field, submitted the payload: `<script>alert('You're HACKED')</script>` 3) Clicked Submit — the script executed immediately, popping a JavaScript alert box reading "HACKED", with the payload visible directly in the URL query string (`?name=<script>alert('HACKED')...`).



**Risk Level:** Medium — exploitation requires convincing a victim to click a crafted link, but successful exploitation enables session token theft, credential phishing overlays, or redirection to malicious sites.

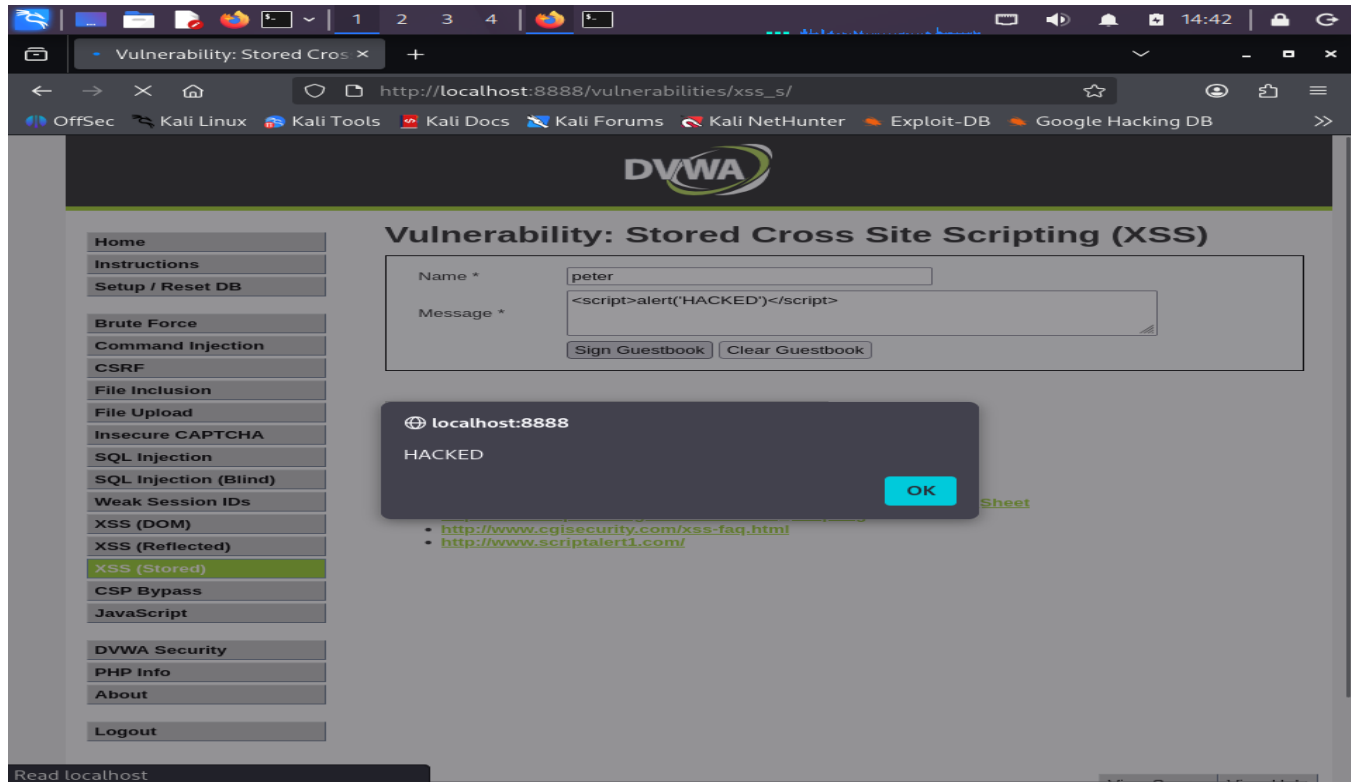
**Fix Recommendation:** Apply context-appropriate output encoding (HTML-entity encode all user-supplied data before rendering), and deploy a Content-Security-Policy header to restrict inline script execution as defense-in-depth.



### Finding 3: Stored Cross-Site Scripting [High]

**Vulnerability:** The DVWA Stored XSS module (/vulnerabilities/xss\_s/) saves guestbook "Message" input directly to the database and renders it unescaped to every visitor of the page.

**Exploitation Steps:** 1) Navigated to the XSS (Stored) module. 2) Entered Name: peter, Message: `<script>alert('HACKED')</script>` 3) Clicked "Sign Guestbook" — the alert fired immediately, and continued to fire on every subsequent page load, confirming the payload was persisted server-side rather than merely reflected once.



**Risk Level:** High — unlike Reflected XSS, no crafted link or social engineering is required; every user who simply visits the guestbook page is automatically compromised, making this the more dangerous of the two variants demonstrated.

**Fix Recommendation:** Sanitize and HTML-encode all user input before storing and again before rendering (encode on output, not just on input). Consider a strict allow-list based sanitizer (e.g. DOMPurify) if any HTML formatting must be preserved, and apply CSP headers as an additional mitigation layer.

## 5.3 Directory & File Enumeration

### Finding 4: Exposed Configuration, Setup, and Version-Control Artifacts [Medium]

**Vulnerability:** A Dirsearch scan against the DVWA instance (now hosted on port 8888) confirmed that several files and directories not intended for production exposure remain directly accessible without authentication.

**Exploitation Steps:** Ran: `dirsearch -u http://127.0.0.1:8888 -o ~/lab/task4/dirsearch/dirsearch-task4.txt` The scan enumerated the following accessible paths (HTTP 200): `/.gitignore`, `/about.php`, `/CHANGELOG.md`, `/README.md`, `/php.ini`, `/robots.txt`, `/favicon.ico`, `/dvwa/`, `/login.php`, and — notably — `/setup.php`, plus directory listings at `/config/` and `/docs/` (both reached via 301 redirects from `/config` and `/docs` respectively).

```

peter@kali: ~/lab/task4
Session Actions Edit View Help
peter@kali: ~ x peter@kali: ~/lab/task4 x
(_|_|_) (/_|_|_|_|_)
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /home/peter/lab/task4/dirsearch/dirsearch-task4.txt
Target: http://127.0.0.1:8888/

[14:44:44] Starting:
[14:44:45] 200 - 57B - /.gitignore
[14:44:46] 403 - 297B - /.ht_wsr.txt
[14:44:46] 403 - 300B - /.htaccess.bak1
[14:44:46] 403 - 300B - /.htaccess.orig
[14:44:46] 403 - 302B - /.htaccess.sample
[14:44:46] 403 - 300B - /.htaccess.save
[14:44:46] 403 - 301B - /.htaccess_extra
[14:44:46] 403 - 300B - /.htaccess_orig
[14:44:46] 403 - 298B - /.htaccessBAK
[14:44:46] 403 - 298B - /.htaccessOLD
[14:44:46] 403 - 299B - /.htaccessOLD2
[14:44:46] 403 - 298B - /.htaccess_sc
[14:44:46] 403 - 290B - /.htm
[14:44:46] 403 - 291B - /.html
[14:44:46] 403 - 300B - /.htpasswd_test
[14:44:46] 403 - 296B - /.htpasswd
[14:44:46] 403 - 297B - /.httr-oauth
[14:44:47] 403 - 290B - /.php
[14:44:47] 403 - 291B - /.php3
[14:44:49] 200 - 2KB - /about.php
[14:44:58] 200 - 7KB - /CHANGELOG.md
[14:44:59] 301 - 314B - /config → http://127.0.0.1:8888/config/
[14:44:59] 200 - 493B - /config/
[14:45:01] 301 - 312B - /docs → http://127.0.0.1:8888/docs/
[14:45:01] 200 - 486B - /docs/
[14:45:02] 200 - 471B - /dvwa/
[14:45:03] 200 - 1KB - /favicon.ico
[14:45:08] 200 - 699B - /login.php
[14:45:13] 200 - 148B - /php.ini
[14:45:16] 200 - 9KB - /README.md
[14:45:17] 200 - 26B - /robots.txt
[14:45:18] 403 - 299B - /server-status
[14:45:18] 403 - 300B - /server-status/
[14:45:19] 200 - 2KB - /setup.php

Task Completed
(peter@kali)-[~/lab/task4]
$

```

**Found Endpoints of Concern:** /setup.php — DVWA's database setup/reset page, which in a misconfigured production deployment could allow an attacker to reset or reconfigure the application database. /config/ and /docs/ — directory listing enabled, exposing internal file structure. /php.ini, /CHANGELOG.md, /README.md, /.gitignore — reveal PHP configuration and version/development history useful for attacker reconnaissance.

**Risk Level:** Medium — no direct data compromise from enumeration alone, but /setup.php being reachable is a meaningful risk if this were a real deployment, and the remaining exposed files materially aid an attacker's reconnaissance and targeting of further attacks.

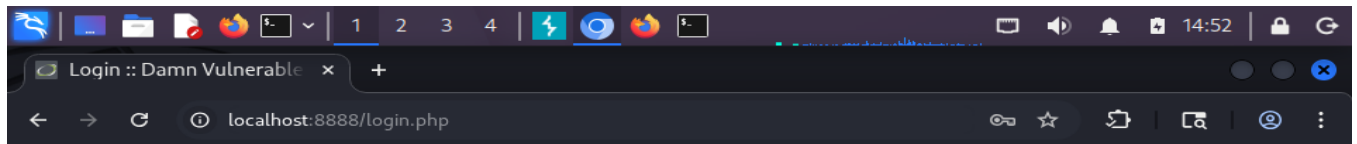
**Fix Recommendation:** Remove or restrict access to /setup.php, development artifacts, and documentation files in any production deployment. Disable directory indexing (Options -Indexes) on all web-accessible directories.

## 5.4 Burp Suite — Interception & Parameter Tampering

### Finding 5: Login Request Interception and Credential Parameter Tampering [Low]

**Vulnerability:** The DVWA login form submits credentials via a standard POST request with no additional client-side or transport protections (e.g. no CSRF-resistant single-use token tied to the credential values, no rate limiting observed) that would prevent a proxy tool from intercepting and freely modifying the submitted values before they reach the server.

**Exploitation Steps:** 1) Attempted a normal login with a random username/password (rajdhani / a random password) — the application returned "Login failed" as expected. 2) Configured Firefox to route traffic through Burp Suite's proxy (127.0.0.1:8080) and enabled Intercept in Burp. 3) Resubmitted the login form — Burp intercepted the raw POST /login.php request, showing the username, password, Login, and user\_token body parameters, along with the security=low session cookie. 4) Sent the intercepted request to Burp Repeater. 5) In Repeater, edited the request body to replace the credential values with DVWA's documented default administrator credentials (username=admin, password=password), then clicked Send. 6) The server responded with HTTP/1.1 302 Found, redirecting away from the login flow — consistent with the login being accepted.



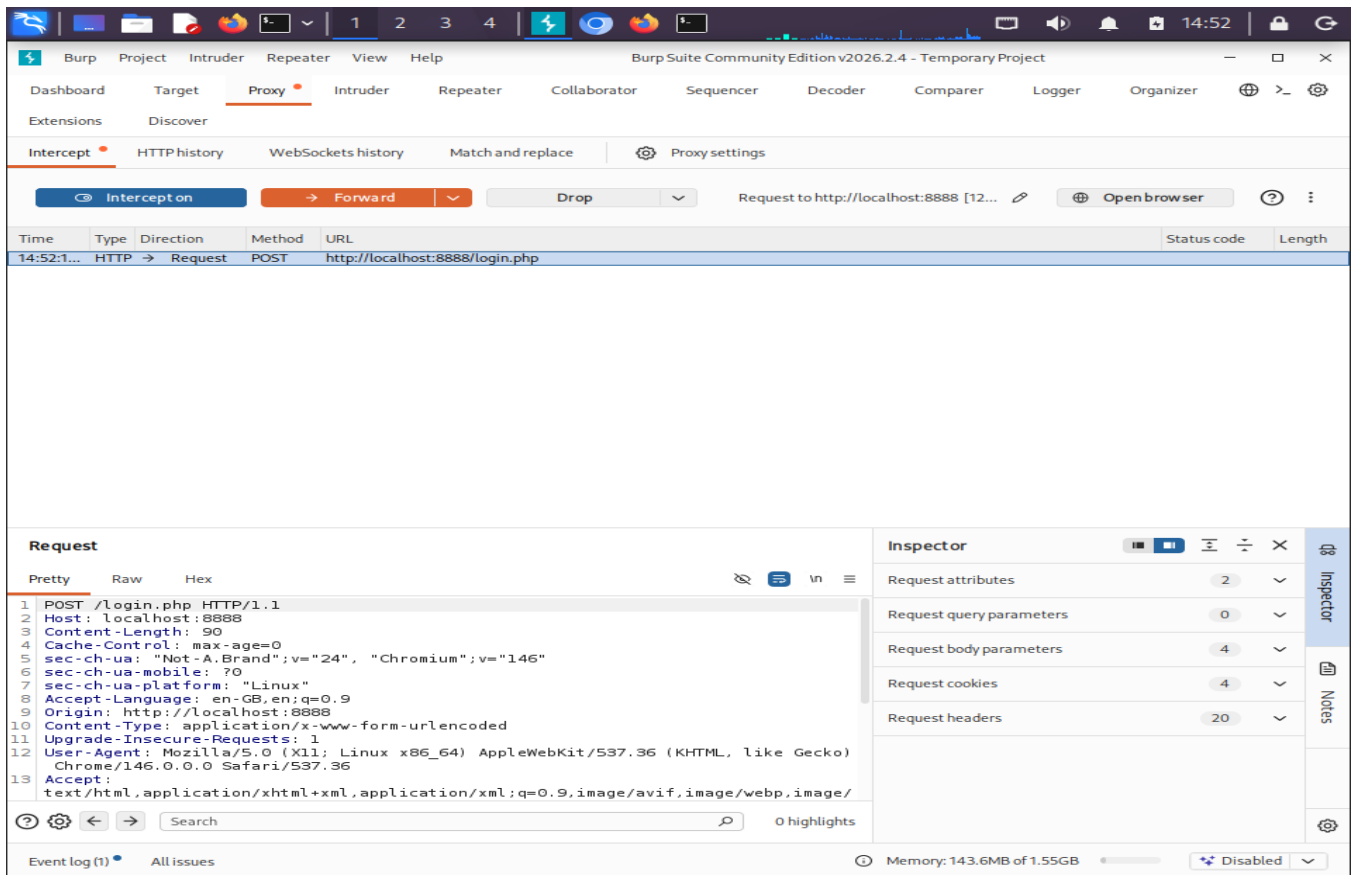
Username

Password

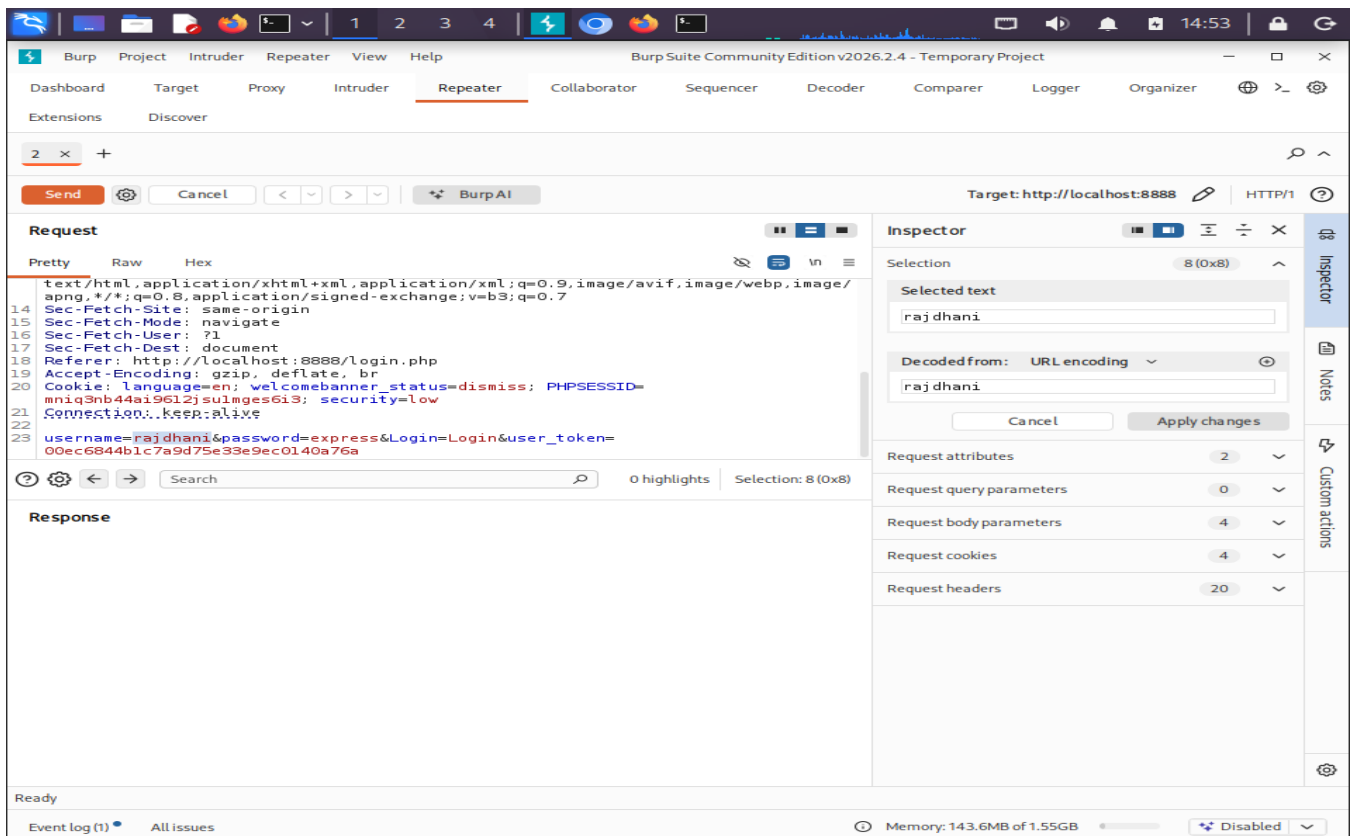
Login failed

[Damn Vulnerable Web Application \(DVWA\)](#)

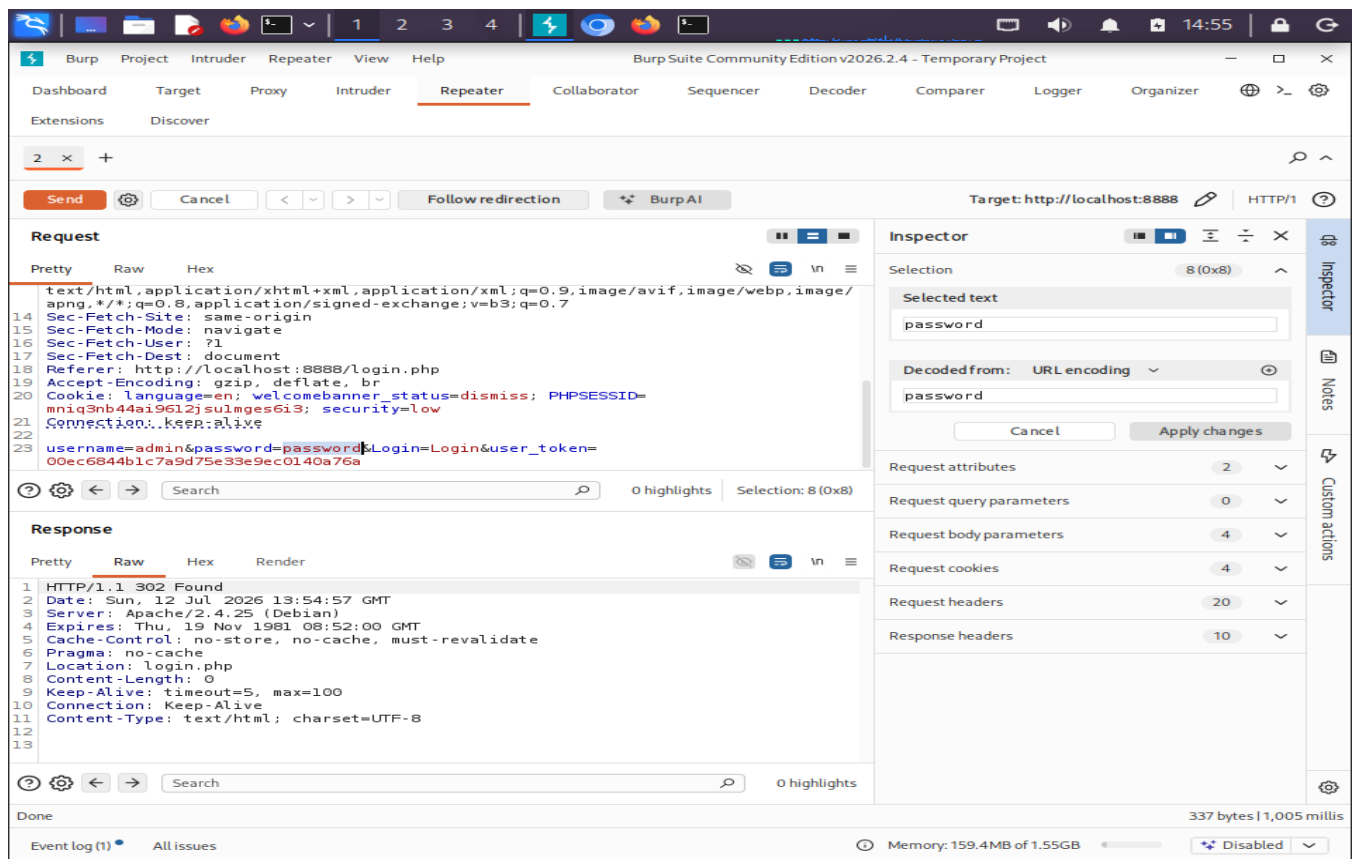
Input Random username & password



Intercept the request using proxy



Request send to the repeater



Modify the and forward the request

**Risk Level:** Low (as demonstrated) — this exercise primarily demonstrates the mechanics of proxy interception and parameter tampering using Burp Suite, which are foundational skills for manual web application testing. The actual risk in this instance stems from the use of a weak, well-known default credential pair rather than a flaw in the interception mechanism itself.

**Fix Recommendation:** Enforce strong, unique credentials and disable or change all default accounts before any deployment. Implement account lockout / rate limiting after repeated failed login attempts, and ensure session tokens are regenerated on privilege change to reduce the value of any intercepted session data.

## 6. Impact Analysis

The table below maps each exploitation finding to its potential real-world security impact if this were a production system rather than an isolated training lab.

| Finding       | Business / Security Impact                                                                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL Injection | Could allow an attacker to bypass authentication entirely or extract the full contents of the users table, including credentials, without needing a valid account. |
| Reflected XSS | Enables session hijacking or credential phishing against any victim who clicks a crafted, attacker-supplied link.                                                  |
| Stored XSS    | Compromises every visitor to the affected page automatically, with no social engineering required — the most severe of the findings in this task.                  |

| Finding                    | Business / Security Impact                                                                                                                                                 |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exposed setup/config files | Aids attacker reconnaissance and, in the case of /setup.php, could allow database reconfiguration if present in a real deployment.                                         |
| Login parameter tampering  | Demonstrates that credentials in transit can be freely intercepted and modified; combined with weak/default credentials, this enables straightforward unauthorized access. |

## 7. Risk Rating Summary

Risk ratings were assigned using a qualitative Critical / High / Medium / Low scale, based on the likelihood of exploitation and potential impact, consistent with standard vulnerability assessment and penetration testing practice (e.g. OWASP risk rating methodology).

| Finding                    | Risk Rating | Justification                                                                                                                       |
|----------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------|
| SQL Injection              | High        | Confirmed, working authentication/query logic bypass with full data enumeration; a well-known, highly damaging vulnerability class. |
| Stored XSS                 | High        | Persistent, self-triggering compromise of any visitor without requiring interaction with a crafted link.                            |
| Reflected XSS              | Medium      | Confirmed script execution, but requires a victim to click a crafted link, lowering practical exploitability versus Stored XSS.     |
| Exposed setup/config files | Medium      | Reconnaissance and configuration-tampering value; no direct data compromise confirmed in this task.                                 |
| Login parameter tampering  | Low         | Demonstrates tooling/technique rather than a distinct application flaw; real risk depends on credential strength.                   |

## 8. Remediation Recommendations

The following remediation steps are recommended for each finding. As DVWA is intentionally vulnerable for training purposes, these recommendations reflect standard industry best practice directly applicable to real production applications built on similar technology stacks.

| Finding       | Recommended Remediation                                                                                                                                    |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQL Injection | Use parameterized queries / prepared statements for all database access; apply least-privilege database accounts and input validation as defense-in-depth. |

| Finding                    | Recommended Remediation                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reflected & Stored XSS     | HTML-encode all user-supplied output; sanitize input where HTML must be preserved (e.g. DOMPurify); deploy a Content-Security-Policy header to restrict inline script execution. |
| Exposed setup/config files | Remove /setup.php and development artifacts from production; disable directory indexing on all web-accessible paths.                                                             |
| Login parameter tampering  | Enforce strong, unique credentials and remove default accounts; add login rate-limiting/lockout; regenerate session tokens on authentication state change.                       |

## 9. Conclusion & Learning Outcomes

---

This task successfully moved from vulnerability identification (Task-3) to hands-on, validated exploitation (Task-4), confirming that the previously identified weaknesses in DVWA are practically exploitable under its default (Low) security configuration. Five findings were documented across SQL Injection, Reflected and Stored XSS, directory/file enumeration, and manual request tampering via Burp Suite, with the highest severity assigned to SQL Injection and Stored XSS due to their direct, self-contained exploitability and impact.

Through this task, the following practical skills were developed:

- Performing real exploitation safely within an isolated lab environment
- Understanding the attacker mindset — how a discovered weakness becomes a working exploit
- Validating vulnerabilities rather than relying on scanner output alone
- Building foundational penetration testing skills, including manual proxy-based testing with Burp Suite
- Producing a professional Penetration Testing Report suitable for stakeholder and client communication

These skills form the foundation for more advanced security testing work, including automated exploitation frameworks, privilege escalation, and full-scope penetration testing engagements.

## Appendix A: Evidence Index

The following evidence files and screenshots accompany this report as a separate evidence folder submission. Insert each screenshot at its corresponding placeholder marked throughout Section 5 of this report.

| Evidence Item               | Description                                                                                                                 |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| dirsearch-task4.txt         | Raw Dirsearch scan output against the DVWA instance on port 8888.                                                           |
| SQL Injection screenshot    | DVWA SQL Injection module showing the ' OR '1'='1 payload and full user table returned.                                     |
| Reflected XSS screenshots   | Payload entry and resulting JavaScript alert box.                                                                           |
| Stored XSS screenshot       | Guestbook entry with payload and persisted alert box firing on page load.                                                   |
| Burp Suite screenshots (x4) | Failed login attempt, intercepted request, Repeater tab with modified username, and final response after modified password. |
| DVWA homepage screenshot    | Confirms the target application and version in scope for this assessment.                                                   |